

VULNERABILITY ASSESSMENT REPORT

**Target: DVWA (Damn Vulnerable Web
Application) — localhost**

Date: April 10, 2026

Target: <http://localhost/dvwa>

IP: 127.0.0.1 (local)

Type: Passive / Non-Destructive

Tools: Nmap · OWASP ZAP · DevTools

Env: Kali Linux VM (VirtualBox)

RISK SUMMARY



1 EXECUTIVE SUMMARY

This report documents the results of a passive security assessment conducted on DVWA (Damn Vulnerable Web Application), an intentionally vulnerable web application installed locally on Kali Linux.

The assessment used standard tools: Nmap 7.99 for network reconnaissance, OWASP ZAP 2.17.0 for passive web vulnerability scanning, and Browser DevTools for manual HTTP header inspection.

A total of 6 vulnerabilities were identified, ranging from Low to High risk. The most critical is a SQL Injection that could allow an attacker to read, modify, or delete the entire database. Immediate remediation is strongly recommended.

2 SCOPE & METHODOLOGY

Target URL — <http://localhost/dvwa>

IP Address — 127.0.0.1 (local machine)

Application — DVWA v2.4 — Vulnerable PHP app (test environment)

Assessment Type — Passive / Non-Destructive

Date — April 10, 2026

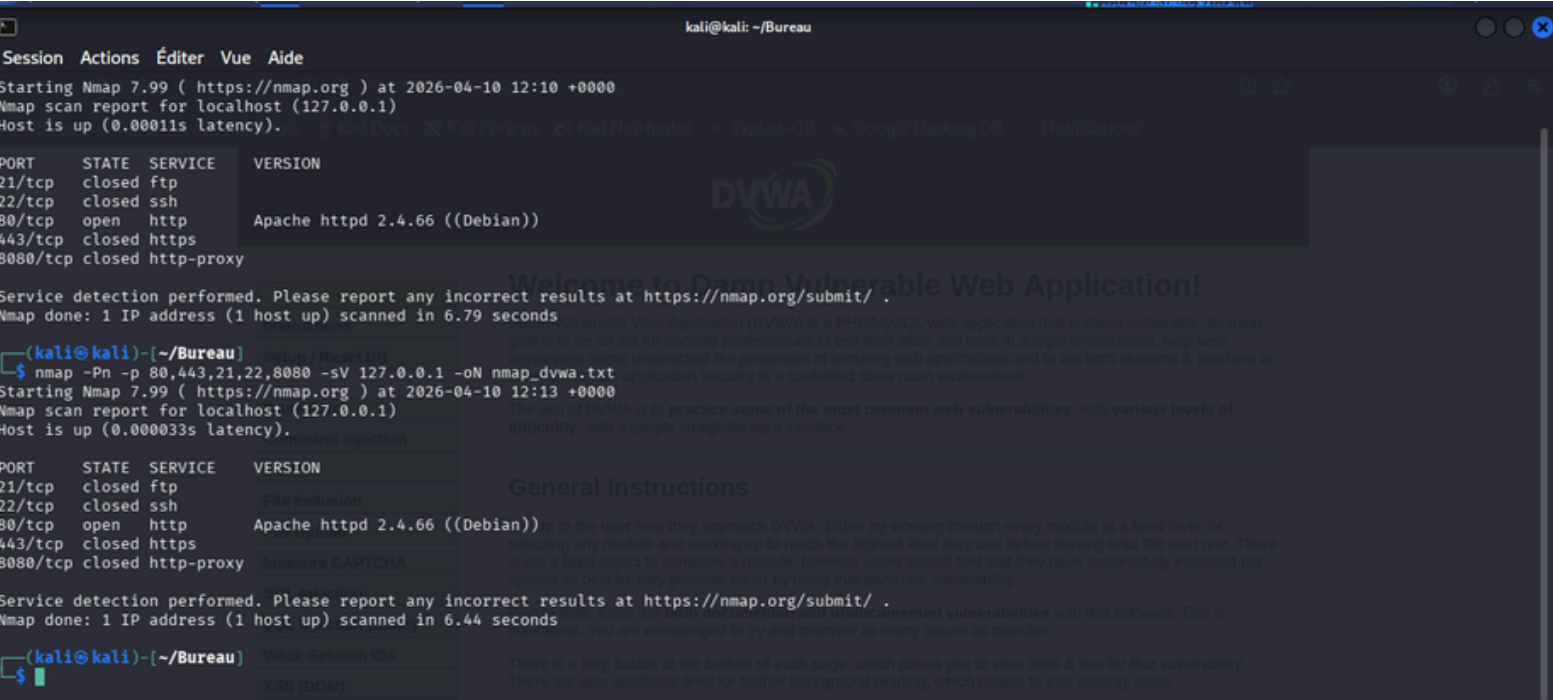
Tools Used:

- Nmap 7.99 — Network port scan and service detection
- OWASP ZAP 2.17.0 — Passive web vulnerability scan via HTTP proxy
- Firefox DevTools — Manual HTTP response header inspection
- Kali Linux 2024.x (VM) — Security assessment environment

3 NETWORK RECONNAISSANCE — NMAP

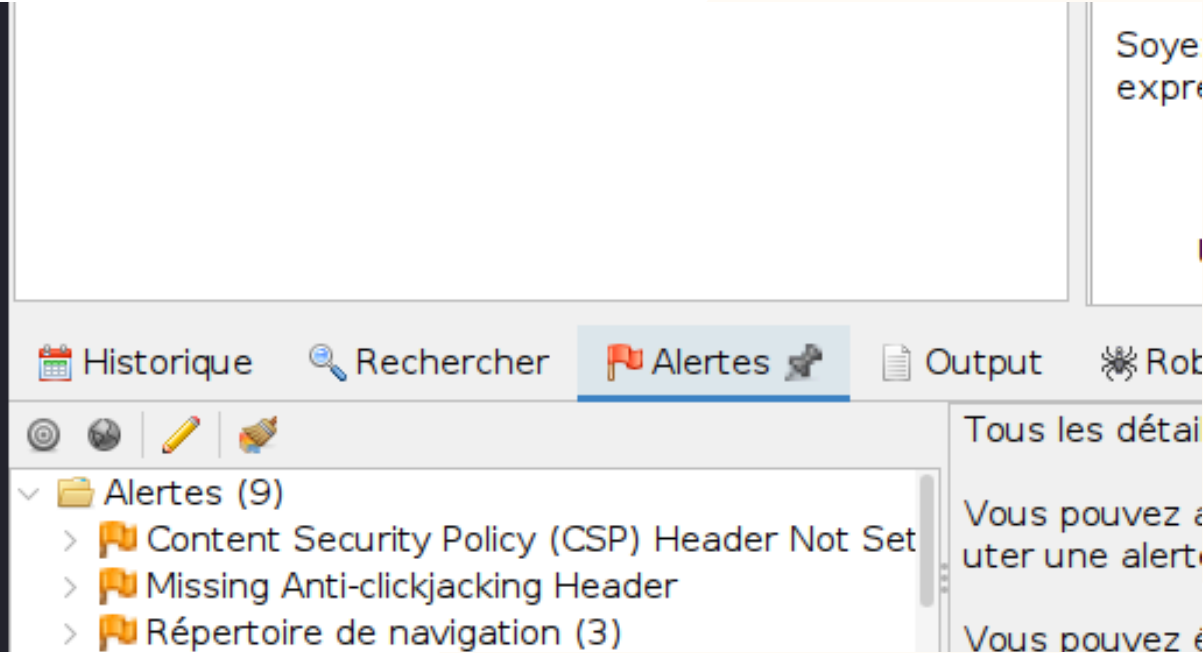
Nmap scan executed against 127.0.0.1 to enumerate open ports and running services.

- Command:** `nmap -Pn -p 80,443,21,22,8080 -sV 127.0.0.1 -oN nmap_dvwa.txt`
- 21/tcpCLOSEDFTP** — Port closed, not in use
 - 22/tcpCLOSEDSSH** — Port closed, not in use
 - 80/tcp OPENHTTP** — Apache httpd 2.4.66 (Debian) — DVWA running
 - 443/tcpCLOSEDHTTPS** — Not configured, no SSL certificate
 - 8080/tcpCLOSEDHTTP-Proxy** — Port closed



4 WEB VULNERABILITY SCAN — OWASP ZAP

Automated scan with OWASP ZAP 2.17.0 on `http://localhost/dvwa/login.php` — 9 alerts detected, grouped into 6 findings below.



FINDING 1 — SQL Injection

HIGH

CWE : CWE-89 — Improper Neutralization of SQL Commands

Instance : <http://localhost/dvwa/vulnerabilities/sqli/>

Description : The application does not sanitize user input before including it in SQL queries. An attacker can inject malicious SQL code to access, modify, or delete sensitive data.

Impact : Complete database compromise — user data, passwords, and private info could be stolen or permanently destroyed.

Fix : Use parameterized queries (prepared statements). Never concatenate user input into SQL strings. Deploy a WAF.

FINDING 2 — Missing Anti-CSRF Tokens

MEDIUM

CWE : CWE-352 — Cross-Site Request Forgery (CSRF)

Instances : 2 affected forms (login page, signup page)

Description : Forms do not include Anti-CSRF tokens. Attackers can trick authenticated users into submitting unauthorized requests without their knowledge.

Impact : Attackers could change passwords or submit actions on behalf of logged-in users.

Fix : Implement Anti-CSRF tokens (synchronizer token pattern) on all forms. Validate server-side on every submission.

FINDING 3 — CSP Header Not Set

MEDIUM

CWE : CWE-693 — Protection Mechanism Failure

Instances : All DVWA pages

Description : No Content-Security-Policy header in HTTP responses. Browsers cannot restrict malicious scripts or external resources from loading.

Impact : Greatly increased XSS risk. Malicious scripts could be injected and run in victims' browsers, leading to session theft.

Fix : Add strict CSP header to all responses. Example: Content-Security-Policy: default-src 'self'; script-src 'self'

FINDING 4 — Missing Anti-Clickjacking Header

MEDIUM

CWE : CWE-1021 — Improper Restriction of Rendered UI Frames

Instances: 2 affected pages

Description : No X-Frame-Options header or frame-ancestors CSP directive. Attackers can embed the site in an invisible iframe for Clickjacking attacks.

Impact : Users could be tricked into clicking invisible elements, causing unintended form submissions or unauthorized transactions.

Fix: Add X-Frame-Options: DENY or SAMEORIGIN to all HTTP responses. Or add frame-ancestors 'none' in CSP.

FINDING 5 — Cookie Without SameSite Attribute

MEDIUM

CWE : CWE-1275 — Sensitive Cookie With Improper SameSite Attribute

Instance : DVWA session cookie

Description : Session cookie lacks the SameSite attribute. This can expose the app to cookie-based CSRF attacks in older browsers.

Impact : An attacker could potentially steal or reuse session cookies of an authenticated user.

Fix : Set SameSite=Strict or SameSite=Lax on all sensitive cookies.

Example: Set-Cookie: PHPSESSID=xxx; SameSite=Strict; HttpOnly; Secure

FINDING 6 — Apache Version Exposed in Headers LOW

CWE : CWE-200 — Exposure of Sensitive Information

Instance : "Server" header in all HTTP responses (DevTools)

Description : HTTP Server header reveals exact Apache version: "Apache/2.4.66 (Debian)". This helps an attacker target version-specific exploits.

Impact : Low risk currently. If a CVE is found for Apache 2.4.66, an attacker would know exactly what to target.

Fix : Configure ServerTokens Prod and ServerSignature Off in Apache config (/etc/apache2/apache2.conf).

Manual inspection of HTTP response headers via Firefox DevTools (Network tab) on <http://localhost/dvwa/login.php>

Content-Security-Policy * MISSING High XSS risk

X-Frame-Options *MISSING Clickjacking risk

X-Content-Type-Options *MISSING MIME sniffing risk

Server *EXPOSED Apache/2.4.66 (Debian) — version visible

Strict-Transport-Security * MISSING No forced HTTPS

The screenshot shows a web browser at <http://localhost/dvwa/login.php>. The page displays the DVWA logo and the text "Demo Vulnerable Web Application (DVWA)". Below the browser window, the Firefox DevTools Network tab is open, showing a list of requests. The selected request is for `login.php`, which is a 1.34 kB HTML document. The response headers for this request are visible on the right side of the DevTools interface.

Status	Method	Domain	File	Initiator	Type	Transferred	Size
200	GET	localhost	favicon.ico	img	NS_BINDING_A...		
200	GET	localhost	login.php	document	html	956 B	1.34 kB
200	GET	localhost	login.css	stylesheet	css	cached	842 B
200	GET	localhost	login_logo.png	img	png	cached	9.09 kB
404	GET	localhost	favicon.ico	FaviconLoader.s...	html	cached	311 B

The response headers for the selected request are:

- Accept-Ranges: bytes
- Connection: Keep-Alive
- Content-Length: 9088
- Content-Type: image/png
- Date: Fri, 10 Apr 2026 12:38:32 GMT
- ETag: "2380-64f1a60a9016"

SQL Injection — Replace all dynamic SQL with parameterized queries (prepared statements) Immediate

Missing Anti-CSRF Tokens — Add synchronizer tokens on all state-changing forms Short term

CSP Header Not Set — Add Content-Security-Policy to all HTTP responses Short term

Missing Anti-Clickjacking Header — Add X-Frame-Options: DENY to all responses Short term

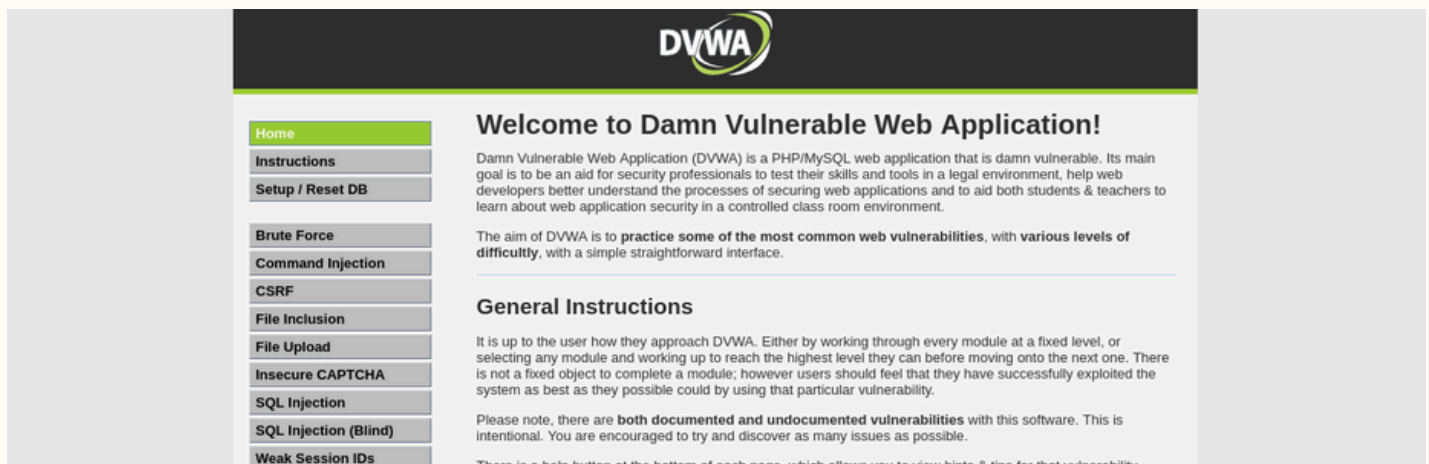
Cookie Without SameSite — Set SameSite=Strict on all session cookies Short term

Apache Version Exposed — Configure ServerTokens Prod in Apache config Long term

CONCLUSION & RECOMMENDATIONS

The security assessment of DVWA revealed 6 vulnerabilities. The most critical is a SQL Injection that represents a direct and immediate threat to data integrity and confidentiality.

- Fix SQL Injection immediately with prepared statements
- Implement Anti-CSRF tokens on all forms
- Add Content-Security-Policy and X-Frame-Options headers
- Configure SameSite=Strict on all session cookies
- Hide Apache version with ServerTokens Prod
- Conduct regular security audits and penetration tests



This assessment was conducted exclusively using passive and non-destructive techniques on an intentionally vulnerable test application (DVWA) installed locally. No production system was targeted or affected.

Prepared as part of the Future Interns — Cyber Security Internship Program (Task 1).